

DOCUMENTO DE PLANTEAMIENTO — SPRINT 0

VALORIA

EL CAMINO DEL HÉROE

Un sistema de hábitos y metas personales convertido en una aventura de rol pixel-art. Cada tarea de tu vida real es una misión. Cada racha, una llama que no debe apagarse. Cada meta grande, un jefe que solo cae tras muchas batallas pequeñas.

PROYECTO ACADÉMICO • FULL STACK

JAVA / SPRING BOOT

POSTGRESQL

REACT

JWT

GOOGLE API

AZURE

Equipo: 2 desarrolladores • Roles rotativos: Scrum Master • Product Owner • Frontend • Backend • Base de Datos • DevOps/Azure

LA PROBLEMÁTICA

La mayoría de aplicaciones de hábitos y productividad son funcionales pero **olvidables**: listas blancas, un ícono minimalista, una barra de progreso gris. Cumplen su función pero no generan ningún vínculo emocional con quien las usa, y por eso se abandonan a los pocos días. El reto de este proyecto es resolver un problema real —la dificultad de mantener hábitos y metas personales de forma constante— con una solución que use mecánicas de videojuego de rol (RPG) para que el usuario sienta que **prograsa como un personaje**, no que simplemente "marca casillas".

VISIÓN DEL PRODUCTO

VALORIA es una plataforma web donde cada usuario crea un personaje que representa su versión en desarrollo personal. Los hábitos y metas que el usuario define en la vida real se convierten en **misiones**. Completarlas otorga experiencia (XP) y oro; mantener la constancia alimenta una **racha** representada como una llama; las metas grandes y de largo plazo se convierten en **jefes** que solo se derrotan completando suficientes misiones relacionadas. El usuario sube de nivel, desbloquea insignias y puede sincronizar sus misiones con su calendario real de Google para no perder de vista sus compromisos.

USUARIOS OBJETIVO

Estudiantes y jóvenes profesionales que quieren construir hábitos (estudio, ejercicio, lectura, finanzas, bienestar) y que disfrutan de la estética de videojuegos retro/RPG. También sirve como caso de estudio realista para un portafolio de ingeniería de software, ya que exige autenticación, modelado de datos no trivial, integración con una API externa y despliegue en la nube.

OBJETIVOS DEL RETO (PARA USTEDES COMO EQUIPO)

- ☐ Vivir un ciclo completo de desarrollo de software tal como se hace en la industria: descubrimiento, diseño, construcción, pruebas y despliegue.
- ☐ Practicar Scrum con roles reales, aunque sean solo dos personas repartiéndoselos.
- ☐ Construir un backend robusto en capas (Spring Boot) con base de datos relacional bien modelada.
- ☐ Integrar un servicio externo real (Google) y desplegar en un proveedor cloud real (Azure).
- ☐ Producir un artefacto de portafolio con identidad visual propia, no una plantilla genérica.



En el capítulo siguiente se definen los roles del equipo y, después, la bitácora de misiones (checklist) que deben ir completando en orden durante el desarrollo.

× **Cap. II — La Compañía**

Son dos aventureros para seis roles. La idea no es que cada uno cargue con tres roles fijos todo el reto, sino que roten *cada sprint* (o cada semana) para que ambos practiquen todas las facetas del desarrollo de software. Aun así, aquí tienen la descripción de cada rol para que sepan qué se espera de quien lo lleve puesto.

SCRUM MASTER Facilita las ceremonias (planning, daily corto, review, retro), elimina obstáculos y cuida que el equipo respete el proceso. No asigna tareas, protege el flujo de trabajo.	PRODUCT OWNER Dueño del backlog. Prioriza qué se construye primero, redacta y afina las Historias de Usuario, y decide cuándo algo está "listo para usarse".
FRONTEND Construye la interfaz en React: pantallas, componentes, consumo de la API, manejo de estado y la estética pixel-RPG del producto.	BACKEND Construye la API en Spring Boot: entidades, lógica de negocio (XP, niveles, rachas), seguridad con JWT y la integración con Google.
BASE DE DATOS Diseña el modelo entidad-relación, define índices y relaciones, cuida la integridad de los datos y las migraciones (Flyway/Liquibase).	DEVOPS / AZURE Configura los recursos en Azure, variables de entorno, pipeline de despliegue (CI/CD) y monitoreo básico de la aplicación en producción.

SUGERENCIA DE REPARTO PARA DOS PERSONAS

Persona A: Scrum Master + Backend + Base de Datos en el Sprint 1, luego rota. Persona B: Product Owner + Frontend + DevOps/Azure en el Sprint 1, luego rota. Así, para cuando termine el reto, ambos habrán tocado los seis roles al menos una vez.

Este es el orden en que se aborda un proyecto real en la industria. No avancen de fase sin cerrar razonablemente la anterior —esa disciplina es parte del aprendizaje.

FASE 0 — PREPARACIÓN DEL EQUIPO (SPRINT 0)

- ☐ Definir roles iniciales y calendario de rotación
- ☐ Crear repositorio (monorepo o repos separados front/back) y convención de ramas
- ☐ Configurar tablero Scrum (Trello, Jira o Azure Boards)
- ☐ Definir Definition of Ready y Definition of Done
- ☐ Definir duración del sprint (sugerido: 1 semana, por ser vacaciones)

FASE 1 — DESCUBRIMIENTO Y REQUISITOS

- ☐ Redactar visión del producto (elevator pitch de VALORIA)
- ☐ Levantar requisitos funcionales (RF) y no funcionales (RNF)
- ☐ Definir roles de usuario dentro de la app (jugador, administrador)
- ☐ Redactar Historias de Usuario: "Como [rol] quiero [acción] para [beneficio]" + criterios de aceptación
- ☐ Priorizar el backlog (MoSCoW o Valor/Esfuerzo)
- ☐ Estimar historias (t-shirt sizing o planning poker simplificado)

FASE 2 — DISEÑO

- ☐ Diagrama de arquitectura general (cliente – API – base de datos)
- ☐ Modelo entidad-relación (usuario, hábito/misión, racha, logro, clase de personaje, nivel)
- ☐ Diagrama de clases del backend (Entity / Repository / Service / Controller)
- ☐ Contrato de API: lista de endpoints REST documentados (OpenAPI/Swagger)
- ☐ Wireframes o mockups del frontend con estilo pixel-RPG (Figma u otra herramienta)
- ☐ Definir sistema de diseño: paleta, tipografías, iconografía pixel

FASE 3 — CONFIGURACIÓN TÉCNICA

- ☐ Crear proyecto Spring Boot (Web, JPA, Security, driver PostgreSQL, Validation)
- ☐ Crear proyecto frontend (React + Vite)
- ☐ Configurar PostgreSQL local y variables de entorno (application.properties / .env)
- ☐ Crear recursos en Azure (App Service, Azure Database for PostgreSQL)
- ☐ Configurar pipeline CI/CD básico (GitHub Actions → Azure)
- ☐ Crear proyecto en Google Cloud Console y habilitar credenciales OAuth2 / Calendar API

FASE 4 — BACKEND

- ☐ Modelar entidades JPA y sus relaciones (@OneToMany, @ManyToOne, etc.)
- ☐ Crear repositorios con Spring Data JPA
- ☐ Implementar lógica de negocio en Services (cálculo de XP, subida de nivel, ruptura de racha)
- ☐ Crear Controllers REST siguiendo el contrato de API definido en Fase 2
- ☐ Implementar registro/login con JWT
- ☐ Implementar autorización por roles
- ☐ Integrar inicio de sesión con Google (OAuth2)
- ☐ Documentar la API (Swagger/OpenAPI)

FASE 5 — FRONTEND

- ☐ Estructurar el proyecto por features (auth, misiones, personaje, mapa)
- ☐ Construir pantallas: login/registro, hoja de personaje, misiones activas, mapa de progreso, perfil
- ☐ Conectar con el backend (axios + manejo de tokens JWT)
- ☐ Manejo de errores y estados de carga
- ☐ Aplicar la estética pixel-RPG (sprites, animaciones CSS, transiciones "de videojuego")

FASE 6 — INTEGRACIONES EXTERNAS

- ☐ Sincronizar misiones con Google Calendar
- ☐ (Opcional) Notificaciones por correo o push cuando una racha está en riesgo
- ☐ (Opcional) Exportar la "hoja de personaje" como imagen o PDF para compartir

FASE 7 — PRUEBAS

- ☐ Probar todos los endpoints con Postman/Insomnia (colección compartida entre el equipo)
- ☐ Pruebas unitarias del backend (JUnit + Mockito)
- ☐ Pruebas de integración de los flujos críticos (registro, login, completar misión)
- ☐ Checklist de pruebas manuales de interfaz (QA)

FASE 8 — DESPLIEGUE

- ☐ Desplegar backend en Azure App Service
- ☐ Desplegar base de datos en Azure Database for PostgreSQL
- ☐ Desplegar frontend (Azure Static Web Apps o similar)
- ☐ Configurar dominio y HTTPS
- ☐ Configurar monitoreo básico (logs / Application Insights)

FASE 9 — CIERRE Y PORTAFOLIO

- ☐ Escribir un README completo (arquitectura, capturas, cómo correr el proyecto)
- ☐ Grabar una demo corta en video
- ☐ Documentar una retrospectiva del proyecto (qué aprendieron, qué mejorarían)
- ☐ Publicar el proyecto en GitHub y en su portafolio / LinkedIn



Cierran el ciclo completo: de la idea al despliegue en producción, tal como se hace en un equipo de desarrollo real.

× Cap. IV – Dirección de Arte

Nada de blanco, minimalismo tímido ni un ícono suelto de color pastel. VALORIA se ve y se siente como un RPG de 16-bits: paleta oscura tipo "mazmorra al anochecer", tipografía pixel para títulos, marcos con esquinas cuadradas doradas, y cada elemento de productividad disfrazado de mecánica de videojuego.

PALETA SUGERIDA

#14131F fondo	#2C2412 panel	#E8B23A oro/XP	#7B2D3D vida/daño	#4CAF7D éxito	#3A6EA5 maná/info
--------------------------	--------------------------	---------------------------	------------------------------	--------------------------	------------------------------

TRADUCCIÓN DE MECÁNICAS (GAMIFICACIÓN)

CONCEPTO REAL	SE CONVIERTE EN...
Hábito o tarea	Misión (Quest)
Constancia diaria	Racha / Llama que crece o se apaga
Categoría del hábito	Clase de personaje (Guerrero=físico, Mago=estudio, Clérigo=bienestar, Pícaro=hábitos rápidos)
Completar tarea	Ganar XP y oro
Progreso acumulado	Subir de nivel, desbloquear cosméticos del personaje
Meta grande a largo plazo	Jefe (Boss) que exige varias misiones para caer
Logro especial	Insignia / trofeo pixel
Panel principal	"Hoja de personaje" (character sheet)
Fallar un hábito	Perder un punto de escudo (con "poción" para perdonar una falla)

NOTAS DE UI

- Tipografía pixel (ej. Press Start 2P) solo para títulos cortos; usar una tipografía legible para párrafos largos.
- Marcos tipo "ventana de diálogo" de RPG: bordes gruesos, esquinas cuadradas, sombras duras (sin blur).
- Animaciones cortas de "level up" y sonido opcional retro (8-bit) al completar misiones.
- Barra de vida/XP con relleno segmentado, no una barra lisa de progreso moderna.

× Cap. V – El Arsenal Técnico

El stack está alineado con lo que ya vienen entrenando (Java + Spring Boot + PostgreSQL + JWT + React), llevado específicamente al caso VALORIA.

CAPA	TECNOLOGÍA	USO ESPECÍFICO EN VALORIA
Backend	Java 17 + Spring Boot	API REST en capas (Entity/Repository/Service/Controller)
Base de datos	PostgreSQL	Usuarios, misiones, rachas, logros, clases, niveles
Seguridad	Spring Security + JWT	Registro/login propio + login con Google (OAuth2)
Frontend	React + Vite	Hoja de personaje, misiones, mapa, animaciones pixel
Integración externa	Google Calendar API	Sincronizar misiones con eventos reales del calendario
Nube	Microsoft Azure	App Service (backend), Azure DB for PostgreSQL, Static Web Apps (frontend)
CI/CD	GitHub Actions	Build y despliegue automático hacia Azure
Documentación API	Swagger/OpenAPI	Contrato de endpoints para pruebas con Postman



En la página final encontrarán el prompt guía adaptado a este proyecto para trabajar con la IA paso a paso durante el desarrollo.

× Anexo — El Grimorio (Prompt guía)

Copien este texto tal cual como primer mensaje al asistente de IA que usen para guiar el desarrollo paso a paso.

```
Actúa como un ingeniero de software senior full stack, especializado en Java con Spring Boot, arquitectura por capas y buenas prácticas profesionales, y guíame en la construcción de un proyecto real llamado VALORIA.

CONTEXTO DEL PROYECTO
VALORIA es una aplicación web de hábitos y metas personales con estética de videojuego RPG pixel-art. Los hábitos del usuario son "misiones", la constancia es una "racha" (llama), las metas grandes son "jefes" que requieren varias misiones para caer, y el usuario sube de nivel y gana XP/oro. La app debe permitir login propio y login con Google, y sincronizar misiones con Google Calendar. El equipo somos dos estudiantes de ingeniería en sistemas (octavo semestre) desarrollando esto en vacaciones como reto de aprendizaje y pieza de portafolio, repartiéndonos roles de Scrum Master, Product Owner, Frontend, Backend, Base de Datos y DevOps/Azure.

STACK A USAR
- Backend: Java 17 + Spring Boot (Web, JPA, Security, Validation)
- Base de datos: PostgreSQL
- Seguridad: Spring Security + JWT + OAuth2 con Google
- Frontend: React (Vite)
- Integración externa: Google Calendar API
- Despliegue: Microsoft Azure (App Service, Azure Database for PostgreSQL, Static Web Apps)

QUIERO QUE ME ENSEÑES ASÍ (sigue este enfoque en todo momento):
1. Explicame primero cómo funciona la arquitectura full stack (frontend, backend, base de datos y cómo se comunican) con ejemplos simples antes de programar.
2. Antes de escribir código, ayúdame a diseñar: requisitos funcionales, entidades del modelo de datos de VALORIA (usuario, misión, racha, clase de personaje, nivel, logro), diagrama de la base de datos, y cómo se vería la arquitectura por capas (Controller, Service, Repository, Entity).
3. Guíame paso a paso en la creación del backend con Spring Boot: crear el proyecto explicando cada opción, la estructura de carpetas, cada capa (Entity, Repository, Service, Controller), cómo funcionan Hibernate y JPA, la conexión con PostgreSQL y application.properties línea por línea.
4. Ayúdame a crear la base de datos en PostgreSQL, entender cómo Hibernate genera las tablas, ver las tablas en pgAdmin, y explicar las relaciones (OneToMany, ManyToOne, etc.) aplicadas al modelo de VALORIA.
5. Implementa conmigo la seguridad: registro/login con JWT explicado desde cero (qué hace cada clase), y después el login con Google vía OAuth2, sin que yo solo copie configuración sin entenderla.
6. Guíame en la creación del frontend en React: estructura de carpetas, pantallas (login, hoja de personaje, misiones, mapa de progreso), conexión con el backend vía axios, manejo de errores, y cómo aplicar una estética pixel-RPG (colores, tipografía, animaciones CSS) sin depender de un framework de diseño genérico tipo Bootstrap plano.
7. Explicame CORS, cómo se prueban los endpoints con Postman antes de conectarlos al frontend, y cómo integrar la sincronización con Google Calendar.
8. Ayúdame con pruebas: uso de Postman, pruebas unitarias con JUnit/Mockito, y errores comunes a detectar.
9. A lo largo de todo el proceso, explicame buenas prácticas: para qué existe cada carpeta, qué genera Spring automáticamente, convenciones de nombres, y cómo se organiza un proyecto real en la industria.
10. Enséñame como si fuera principiante en cada tema nuevo, sin asumir conocimiento previo, avanzando por etapas y sin darme todo el código de una sola vez. Hazme preguntas para confirmar que entendí antes de avanzar, y señala errores comunes y cómo evitarlos.

Quiero aprender de verdad, no solo que el proyecto funcione. Empecemos por el punto 1: explicame la arquitectura full stack aplicada a VALORIA.
```